

Vježba 17

Pytorch - Definiranje vlastitog skupa podataka



Sadržaj

- ImageFolder
- Definiranje vlastitog skupa podataka (Custom Dataset)
 - Što je `torch.utils.data.Dataset`?
- Vlastiti dataset za klasifikaciju
- Vlastiti dataset za detekciju

ImageFolder

ImageFolder

`torchvision.datasets.ImageFolder` je gotova implementacija `Dataset` klase za klasifikaciju slika.

Koristi se kada su podaci organizirani ovako:

```
root/  
  class_0/  
    img1.jpg  
    img2.jpg  
  class_1/  
    img3.jpg  
    img4.jpg
```

Naziv foldera = naziv klase.

Što radi interno?

- Prolazi kroz root folder
- Svaki podfolder tretira kao klasu
- Automatski radi mapiranje:

```
{  
  "Cat": 0,  
  "Dog": 1  
}
```

ImageFolder

Kada koristiti ImageFolder?

Koristi se kada:

- ✓ radiš klasifikaciju
- ✓ svaka slika pripada točno jednoj klasi
- ✓ podaci su organizirani po folderima
- ✓ nema dodatnih metapodataka (CSV, JSON, itd.)

```
from torchvision.datasets import ImageFolder
```

```
dataset = ImageFolder(root="PetImages", transform=transform)
```

Definiranje vlastitog skupa podataka (Custom Dataset)

Definiranje vlastitog skupa podataka (Custom Dataset)

Cilj današnje vježbe: naučiti kako u PyTorchu učitati “realne” podatke koji nisu u standardnom formatu.

U praksi često imamo:

- slike u jednom folderu (images/)
- anotacije u zasebnoj datoteci (labels.csv)
- dataset nije organiziran po klasama u podfolderima (tako da ImageFolder ne pomaže)

Što PyTorch zapravo treba?

Dataset zna:

- koliko uzoraka ima (`__len__`)
- kako dohvatiti jedan uzorak (`__getitem__`)

DataLoader zatim radi:

- batching, shuffle i paralelno učitavanje

Tok podataka kroz kod:

Disk (slike + CSV) → Dataset → DataLoader → trening petlja → model

Definiranje vlastitog skupa podataka (Custom Dataset)

Dataset je **apstraktna klasa** koju nasljeđujemo kada želimo definirati vlastiti način učitavanja podataka.

To znači:

- PyTorch ne zna ništa o našem formatu podataka
- mi definiramo kako se jedan uzorak učitava i vraća

Koju klasu nasljeđujemo?

```
from torch.utils.data import Dataset
```

```
class MojDataset(Dataset):
```

```
    ...
```


Definiranje vlastitog skupa podataka (Custom Dataset)

Koje metode MORAMO implementirati?

1) `__init__(self, ...)`

- učitava metapodatke (npr. CSV)
- priprema liste putanja i labela
- definira transformacije

Ovdje NE učitavamo sve slike u memoriju.

2) `__len__(self)`

- vraća ukupan broj uzoraka
- omogućuje PyTorchu da zna veličinu dataseta

3) `__getitem__(self, idx)`

- prima indeks
- učitava jednu sliku s diska
- vraća:
 - image (tensor)
 - label (tensor)

Ključna ideja

Dataset mora omogućiti:

```
image, label = dataset[0]
```

Ako to radi ispravno → sve ostalo (DataLoader, trening) će raditi.

Definiranje vlastitog skupa podataka (Custom Dataset)

Predložak Custom Dataset klase

```
class CustomImageDataset(Dataset):  
  
    def __init__(self, csv_path, images_dir, transform=None):  
        """  
        Ovdje:  
        - učitavamo CSV  
        - spremamo putanje do slika  
        - radimo label encoding  
        - spremamo transformacije  
        """  
        pass  
  
    def __len__(self):  
        """  
        Mora vratiti ukupan broj uzoraka u datasetu.  
        """  
        pass  
  
    def __getitem__(self, idx):  
        """  
        Mora:  
        - učitati sliku prema indeksu  
        - primijeniti transformacije  
        - vratiti (image_tensor, label_tensor)  
        """  
        pass
```

Vlastiti dataset za klasifikaciju

Vlastiti dataset za klasifikaciju

Dataset za klasifikaciju: Što se očekuje?

Kod klasifikacije svaki uzorak predstavlja:
jedna slika → jedna labela

Metoda `__getitem__(idx)` mora vratiti:
(image, label)

Tip i oblik povratnih vrijednosti

image

- tip: torch.Tensor
- dtype: torch.float32
- shape: [C, H, W]
- vrijednosti: najčešće u rasponu [0,1] (nakon ToTensor())

Primjer:

[3, 224, 224]	# RGB slika
[1, H, W]	# grayscale slika

label

- tip: torch.Tensor
- dtype: torch.long
- shape: [] (scalar) ili [1]
- vrijednost: indeks klase (0,1,2,...)

Primjer: `tensor(1)`

Vlastiti dataset za klasifikaciju

Gdje i kako se primjenjuju transformacije?

1 Transformacije se definiraju izvan Dataset-a

```
transform = transforms.Compose([  
    transforms.Resize((224, 224)),  
    transforms.ToTensor(),  
    transforms.Normalize(mean, std)  
])
```

Vlastiti dataset za klasifikaciju

Gdje i kako se primjenjuju transformacije?

2 Prosljeđuju se kroz konstruktor

```
dataset = CustomImageDataset(  
    csv_path="labels.csv",  
    images_dir="images",  
    transform=transform  
)
```

Vlastiti dataset za klasifikaciju

Gdje i kako se primjenjuju transformacije?

③ Primjenjuju se u `__getitem__`

```
def __getitem__(self, idx):  
    image = Image.open(path).convert("RGB")  
  
    if self.transform:  
        image = self.transform(image)  
  
    return image, label
```

Vlastiti dataset za detekciju

Vlastiti dataset za klasifikaciju

Što se mijenja kod Dataset-a za detekciju?

Kod klasifikacije: 1 slika $\rightarrow$ 1 labela

Kod detekcije: 1 slika $\rightarrow$ **N objekata**

gdje je N različit za svaku sliku.

To znači da `__getitem__` više ne vraća (image, label) nego:
(image, target_dict)

Vlastiti dataset za klasifikaciju

Očekivani izlaz kod detekcije

image

- tip: torch.Tensor
- dtype: torch.float32
- shape: [C, H, W]
- tipično: samo ToTensor() (bez resize, osim ako skaliramo i boxeve)

target (dict)

Mora sadržavati barem:

```
target = {  
    "boxes": FloatTensor [N, 4],  
    "labels": LongTensor [N]  
}
```

boxes

- tip: torch.float32
- shape: [N, 4]
- format: [xmin, ymin, xmax, ymax]
- koordinate u pikselima

labels

- tip: torch.int64
- shape: [N]
- vrijednosti: id klase (1,2,3,...)



Konvencija:

- 0 = background
- klase počinju od 1

Vlastiti dataset za klasifikaciju

Zašto više ne vraćamo samo Tensor?

Jer broj objekata po slici nije konstantan.

Primjeri:

- Slika A → 1 pješak
- Slika B → 5 pješaka
- Slika C → 0 pješaka

Shape boxes se mijenja po slici.

Zbog toga:

- ne možemo koristiti standardni batch stacking
- moramo koristiti `collate_fn`

Vlastiti dataset za klasifikaciju

Transformacije kod detekcije

Kod klasifikacije možemo slobodno:

- Resize
- Crop
- Flip
- itd.

Kod detekcije:

 Ako mijenjamo sliku, moramo mijenjati i bounding boxeve.

Primjer:

Ako skaliramo sliku $\times 0.5$

→ moramo skalirati i koordinate boxeva $\times 0.5$

Zato za zadatak:

`transforms.ToTensor()`

bez resize operacija.

Vlastiti dataset za klasifikaciju

Problem batchiranja kod detekcije

Što DataLoader radi kod klasifikacije?

Ako Dataset vraća (image, label)

DataLoader napravi:

```
images = torch.stack([...]) # [B, C, H, W]
```

```
labels = torch.stack([...]) # [B]
```

To radi jer su svi tensor-i iste dimenzije.

Vlastiti dataset za klasifikaciju

Problem batchiranja kod detekcije

Što se događa kod detekcije?

Dataset vraća (image, target_dict)

gdje:

`boxes.shape = [N, 4]`

Ali N je različit za svaku sliku!

DataLoader pokušava napraviti:

`torch.stack([boxesA, boxesB, boxesC]) -> nemoguće, runtime error`

Vlastiti dataset za klasifikaciju

Problem batchiranja kod detekcije

Rješenje: `collate_fn`

Što je `collate_fn`?

Funkcija koja kaže DataLoader-u:

"Evo lista uzoraka — ti odluči kako ih spojiti u batch."

Bez `collate_fn` koristi se default ponašanje (stackanje).

Kod detekcije mi to mijenjamo.

Minimalna implementacija

```
def detection_collate_fn(batch):  
    images, targets = zip(*batch)  
    return list(images), list(targets)
```

Vlastiti dataset za klasifikaciju

Problem batchiranja kod detekcije

Što sada dobivamo u trening petlji?

Umjesto:

```
images.shape = [B, C, H, W]
```

dobivamo:

```
images = [img0, img1, img2]
```

```
targets = [target0, target1, target2]
```


Vlastiti dataset za klasifikaciju

Problem batchiranja kod detekcije

Zašto detekcijski modeli očekuju listu?

Faster R-CNN (i ostali TorchVision modeli) očekuju:
`model(images, targets)`

gdje je:

`images` = list[Tensors]

`targets` = list[dict]

Zašto?

- Svaka slika može imati različitu rezoluciju
- Svaka slika ima različit broj bounding boxeva
- Model interno prolazi kroz listu

Pitanja ?

Slijedi rješavanje zadataka